توضیح خط‌به‌خط و دقیق فایل script.js

کد کامل:

```
const hourHand = document.querySelector(".hour");
const minuteHand = document.querySelector(".minute");
const secondHand = document.querySelector(".second");

function updateClock() {
const now = new Date();
const milliseconds = now.getMilliseconds();
const seconds = now.getSeconds() + milliseconds / 1000;
const minutes = now.getMinutes() + seconds / 60;
const hours = (now.getHours() % 12) + minutes / 60;

hourHand.style.transform = `translateX(-50%) rotate(${hours * 30}deg)`;
minuteHand.style.transform = `translateX(-50%) rotate(${minutes * 6}deg)`;
secondHand.style.transform = `translateX(-50%) rotate(${seconds * 6}deg)`;

requestAnimationFrame(updateClock);
}

updateClock();
```

توضیح خط‌به‌خط

خط ۱ تا ۳:
با document.querySelector سه المان با کلاس‌های hour، minute و second از HTML انتخاب می‌شوند.
هدف این است که مستقیم به عقربه‌ها دسترسی داشته باشیم.

خط ۵: تابع updateClock
این تابع هر بار زمان را حساب می‌کند و زاویه هر عقربه را تغییر می‌دهد.

خط ۶: new Date()
زمان دقیق حال حاضر را برمی‌گرداند. شامل ساعت، دقیقه، ثانیه و میلی‌ثانیه.

خط ۷: getMilliseconds()
مقدار میلی‌ثانیه را بین ۰ تا ۹۹۹ می‌گیرد.

خط ۸: seconds = seconds + milliseconds/1000
مثال: اگر الان ۳۰ ثانیه و ۵۰۰ میلی‌ثانیه گذشته باشد، مقدار ۳۰٫۵ می‌شود.
با این کار عقربه ثانیه‌شمار کاملا نرم و پیوسته حرکت می‌کند.
در غیر این صورت هر ثانیه یک پرش می‌داشت.

خط ۹: minutes = minutes + seconds/60
با این کار عقربه دقیقه کم‌کم حرکت می‌کند نه اینکه دقیقا در لحظه عوض شدن دقیقه بپرد.
مثال: دقیقه ۵ و ثانیه ۳۰ => دقیقه ۵٫۵ => عقربه دقیقا وسط بین ۵ و ۶ است.

خط ۱۰: hours = (hours % 12) + minutes/60
ساعت آنالوگ ۱۲ ساعته است. اگر ساعت ۱۴ باشد، ۱۴ % ۱۲ = ۲ می‌شود.
با اضافه کردن minutes/60، عقربه ساعت هم خیلی آرام و طبیعی حرکت می‌کند.

خط ۱۲ تا ۱۴: چرخش عقربه‌ها
هر ساعت: ۳۶۰ ÷ ۱۲ = ۳۰ درجه => hours * 30
هر دقیقه: ۳۶۰ ÷ ۶۰ = ۶ درجه => minutes * 6
هر ثانیه: ۳۶۰ ÷ ۶۰ = ۶ درجه => seconds * 6

translateX(-50%)
قبل از rotate، عقربه را طوری جابجا می‌کند که نقطه چرخشش دقیقا روی مرکز دایره بیفتد.
اگر نباشد، عقربه حول گوشه خودش می‌چرخد.

خط ۱۶: requestAnimationFrame(updateClock)
این متد تابع را همزمان با رفرش صفحه صدا می‌زند.
برخلاف setInterval، سریع‌تر، روان‌تر و بهینه‌تر است.
اگر تب غیرفعال شود، مرورگر اجرای آن را موقتا متوقف می‌کند.

خط ۱۹: updateClock()
برای بار اول تابع فراخوانی می‌شود تا ساعت بلافاصله نمایش داده شود.
سپس داخل تابع، requestAnimationFrame دوباره آن را صدا می‌زند و این چرخه ادامه پیدا می‌کند.

جمع‌بندی
ایده ساده است: زمان را با دقت محاسبه کن، به درجه تبدیل کن و با transform بچرخان.
فرق این ساعت با حالت ساده این است که با استفاده از میلی‌ثانیه، حرکت کاملا پیوسته و طبیعی شده است.